

Bioinformatics setup workshop manual

Start here	2
Official downloads	2
Module 1 — Ubuntu on Windows (WSL)	2
Module 2 — Prepare a Mac	3
Module 3 — Miniforge and a conda environment	3
Module 4 — Install and configure Git	4
Module 5 — Create a GitHub account	5
Module 6 — Install VS Code and publish a practice repository	5
If something fails	5
Sources	6

Windows/Ubuntu (WSL), Mac, and conda — a hands-on guide for beginners.

Start here

- On Windows: do Module 1 (WSL), then the **Ubuntu route** in Module 3 and its common exercise. The workshop environment's bioinformatics tools come from bioconda, which runs only on Linux or macOS, so build it inside WSL Ubuntu — not the native Windows Miniforge Prompt.
- On Mac: do Module 2, the Mac route in Module 3, and the common conda exercise.
- After Module 3, use Modules 4–6 to set up Git, GitHub, and VS Code; GitHub publishing is optional.
- Bring internet access and permission to install software. WSL requires a Windows restart. Ask IT before changing an institution-managed computer.

A terminal is where you type a command and press Enter. PowerShell, the Ubuntu (WSL) Terminal, and Mac Terminal are different; use the terminal named in each step, and on Windows the workshop environment runs in the Ubuntu (WSL) terminal. Run commands one line at a time, especially when an installer asks questions. If a command fails, stop and ask the instructor.

Quick terms: `~` is your Ubuntu/Mac home folder; `cd` changes folder; `pwd` shows the current folder. A conda environment keeps one project's software separate.

Official downloads

Choose only the files for your route. Do not use third-party download sites.

- WSL + Ubuntu: `wsl --install` downloads both; Microsoft guide: <https://learn.microsoft.com/en-us/windows/wsl/install>
- Windows Miniforge (`.exe`): https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-Windows-x86_64.exe
- Ubuntu Miniforge, `x86_64` (`.sh`): https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-Linux-x86_64.sh
- Ubuntu Miniforge, `aarch64` (`.sh`): <https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-Linux-aarch64.sh>
- Mac Miniforge, Apple silicon `arm64` (`.pkg`): <https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-MacOSX-arm64.pkg>
- Mac Miniforge, Intel `x86_64` (`.pkg`): https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-MacOSX-x86_64.pkg
- Git downloads (Windows Git for Windows; macOS options): <https://git-scm.com/downloads>
- VS Code for Windows or Mac: <https://code.visualstudio.com/download>
- GitHub account: <https://github.com/signup>
- Workshop software (Biopython, pandas, PyMOL, and the bioinformatics tools): conda downloads them in Module 3; no separate installers are needed.

Module 1 — Ubuntu on Windows (WSL)

Windows 11 or Windows 10 build 19041+ supports the standard installation [1]. To display Linux GUI programs in WSL, use Windows 11 or Windows 10 build 19044+ and WSL 2 [2]. Check the build under Settings → System → About.

1. Search for PowerShell, select “Run as administrator,” then run:

```
wsl --install
```

Restart Windows when prompted. If WSL already exists and this only shows help, run `wsl --list --online`, then `wsl --install -d Ubuntu` in PowerShell [1].

1. Open Ubuntu from Start. Create a Linux username and password when asked. Password characters will not appear as you type. In a separate PowerShell window, run:

```
wsl --list --verbose
```

Check that Ubuntu appears with version 2. In the Ubuntu terminal (not PowerShell), run:

```
uname -m
sudo apt update
sudo apt install -y curl ca-certificates git
mkdir -p ~/workshop
cd ~/workshop
pwd
```

`uname -m` shows `x86_64` or `aarch64`, which selects the Linux Miniforge installer. `sudo` may ask for your Ubuntu password. `pwd` should show a path under `/home/`. Keep Linux projects under `~/workshop`, not `/mnt/c/`, when practical [3].

Checkpoint: PowerShell lists Ubuntu as WSL 2; Ubuntu's `pwd` begins with `/home/`.

Module 2 — Prepare a Mac

Open Terminal using Spotlight. Enter:

```
sw_vers
uname -m
mkdir -p ~/workshop/data ~/workshop/results
cd ~/workshop
pwd
```

`sw_vers` shows macOS; current Miniforge needs macOS 11+ [4]. Write down `uname -m`: `arm64` means Apple silicon, `x86_64` means Intel. Choose the matching Mac installer above. `pwd` shows your workshop folder; keep original data in `data` and outputs in `results`. Do not replace macOS's system Python. Apple's Command Line Tools are optional for this workshop; if needed later, use `xcode-select --install` and follow the Apple prompt.

Checkpoint: you know your Mac architecture and can open `~/workshop`.

Module 3 — Miniforge and a conda environment

Miniforge supplies conda, which installs software in isolated environments [4, 5]. Pick ONE installation route below, then do the common exercise. If conda is already installed and working, ask the instructor before installing a second copy.

Windows route

The workshop environment's bioinformatics tools (from bioconda) do not run on native Windows, so build the environment inside WSL Ubuntu: install Miniforge with the **Ubuntu/WSL route** below and do the common exercise there. The native Windows Miniforge `.exe` from Official downloads is optional and only for general Windows conda practice or Windows-native PyMOL later; it cannot build this workshop environment.

Ubuntu/WSL route

In Ubuntu Terminal (never PowerShell), download and install the Linux version matching `uname -m`:

```
cd ~
base=https://github.com/conda-forge/miniforge/releases/latest/download
curl -fL -o Miniforge3.sh "$base/Miniforge3-Linux-$(uname -m).sh"
ls -lh Miniforge3.sh
bash Miniforge3.sh
```

Stop if the download fails; do not run a missing file. Read and accept the license, install in your Linux home, and answer `yes` to shell initialization. Close and reopen Ubuntu. `conda --version` should now work. If you skipped initialization at the default location, run `~/miniforge3/bin/conda init bash` and reopen Ubuntu [4].

Mac route

Run ONE `.pkg` from Official downloads matching your `uname -m`. Choose shell initialization if offered; close and reopen Terminal. Check `conda --version`. Use Miniforge's official installer rather than a Homebrew repackaging [4].

Common exercise (Ubuntu/WSL or Mac)

The workshop uses a single environment, `protein_modeling`, that holds every tool the later sessions need. Build it in your reopened Ubuntu (WSL) or Mac terminal. If your instructor has given you the Ologist_Workshop project folder, the setup script builds it in one step:

```
bash setup_conda_envs.sh
```

If you do not have the project folder yet, run the same command the script runs. Wait for conda's question and type `y` to approve; the bioconda tools make this a larger install than a two-package environment, so give it a few minutes:

```
conda create -n protein_modeling \
  -c conda-forge -c bioconda python=3.11 \
  biopython pandas pymol-open-source \
  fastqc seqkit bwa samtools bcftools blast \
  vina meeko rdkit
```

Either way, activate it and test it:

```
conda activate protein_modeling
python --version
python -c "import Bio, pandas, pymol; print('biopython', Bio.__version__, 'pandas',
pandas.__version__)"
conda info --envs
```

Look for `(protein_modeling)` in your prompt, Python `3.11`, the two package version numbers, and a `*` beside the active environment. Use `conda deactivate` to leave it. Never use `sudo conda` or install workshop packages into `(base)`. If the environment already exists, activate it rather than creating it again — the setup script skips it automatically.

Checkpoint: show the active environment and successful package-version test to a partner.

Module 4 — Install and configure Git

Git records local file history; GitHub is an optional online host. Install Git in the **same system as your project** [6, 7]:

- Windows-native project: install Git for Windows from Official downloads. Open **Git Bash** for the commands below. If you work only in Ubuntu/WSL, you already installed Linux Git in Module 1; a second Windows Git installation is not required.
- Ubuntu/WSL: in Ubuntu Terminal run `git --version`. If Git is missing, run `sudo apt update` and `sudo apt install -y git`.
- Mac: in Terminal run `git --version`. If Git is missing or macOS asks for developer tools, run `xcode-select --install`, finish the Apple prompt, reopen Terminal, and check again.

In Git Bash, Ubuntu Terminal, or Mac Terminal, replace the example identity with **your own** name and a verified GitHub email address (or your GitHub-provided no-reply address if you want to keep your email private). These values appear in published commit history; they are not login credentials [7].

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global --get user.name
git config --global --get user.email
git --version
```

Do not copy the example name/email unchanged. `--global` configures only the Git installation for that operating system/user; Windows Git and WSL Git have separate settings. Checkpoint: `git --version` and both `--get` commands show your intended values.

Module 5 — Create a GitHub account

Visit <https://github.com/signup>, create an account, verify the email address, and follow GitHub's security prompts [8]. Enable two-factor authentication if it is not already required. In GitHub **Settings → Emails**, choose whether to keep your address private; use the address you chose in Module 4 for commit attribution. Keep passwords, recovery codes, and tokens out of files and Git commits. An account is not required for local Git practice, but is required to publish to GitHub.

Checkpoint: you can sign in at `github.com` in a browser; do not share your credentials with the instructor.

Module 6 — Install VS Code and publish a practice repository

Install **Visual Studio Code** from Official downloads on Windows or Mac [9]. For Ubuntu/WSL projects, install VS Code on **Windows, not inside WSL**, then install Microsoft's **WSL** extension in VS Code [10]. From Ubuntu Terminal, `cd ~/workshop`, then `code .`; check for **WSL: Ubuntu** at the bottom left of the VS Code window. Windows-native and Mac users can instead use **File → Open Folder** to open a local folder. Use VS Code's **Terminal → New Terminal** and run `git --version`; if it fails, restart VS Code after installing Git.

Practice in a **new, empty** `git-practice` folder (not an existing research project). On Ubuntu/Mac, create it with `mkdir -p ~/workshop/git-practice`; on Windows, create it using File Explorer. Open that folder in VS Code (WSL: `cd ~/workshop/git-practice` then `code .`). Only trust a folder you created or know is safe.

1. In VS Code, create `README.md`, type `# Git practice`, and save it. Open **Source Control** (branch icon; `Ctrl+Shift+G` on Windows/Linux or `Cmd+Shift+G` on Mac). Select **Initialize Repository**. This creates a *local* repository; nothing is uploaded [11].
2. Review the changed file. Stage **only** `README.md` with its `+` button, enter a message such as `Add workshop README`, and select **Commit**. In the integrated terminal, `git status` should show a clean working tree and `git log -1 --oneline` should show the commit.
3. Optional: in Source Control select **Publish Branch / Publish to GitHub**. Choose a **private** repository for the exercise; follow VS Code's browser sign-in prompt and confirm the proposed files and visibility before publishing [12]. Open the resulting GitHub repository URL in your browser and confirm `README.md` is there. Future commits remain local until you **Push**. If a push fails, stop; do not force-push or repeatedly sync.

Do not publish patient data, unpublished research data, passwords, `.env` files, tokens, or large raw sequencing files. Check your institution's data-sharing rules first. VS Code has built-in Git and GitHub sign-in for this exercise; the GitHub Pull Requests extension is optional [12].

Checkpoint: show your local commit; if you chose to publish, show the **private** GitHub repository and its `README.md` without displaying credentials.

If something fails

- `wsl` unavailable: use PowerShell, check your Windows build [1]. If installation stalls at 0%, Microsoft suggests `wsl --install --web-download -d Ubuntu` [1].
- `conda` unavailable: reopen the terminal and check shell initialization [4]. On Windows, use your WSL Ubuntu terminal, not PowerShell or the native Miniforge Prompt.
- Permission or download error: work in your home folder. Do not disable security controls or use `sudo conda .`
- Git unavailable in VS Code: confirm installation in the same OS/WSL terminal, then restart VS Code [10, 11].

- GitHub publish/sign-in fails: check that your email is verified and follow the browser prompt; confirm you can access the repository. Do not paste passwords or tokens into the workshop document or use force-push [8, 12].

Sources

[1] Microsoft WSL installation: <https://learn.microsoft.com/en-us/windows/wsl/install>

[2] Microsoft WSL GUI apps: <https://learn.microsoft.com/en-us/windows/wsl/tutorials/gui-apps>

[3] Microsoft WSL environment: <https://learn.microsoft.com/en-us/windows/wsl/setup/environment>

[4] Miniforge installation: <https://github.com/conda-forge/miniforge/blob/main/README.md>

[5] Conda environments: <https://docs.conda.io/projects/conda/en/stable/user-guide/tasks/manage-environments.html> [6] Git

downloads: <https://git-scm.com/downloads> [7] Git configuration: <https://git-scm.com/book/en/v2/Getting-Started-First-Time-Git-Setup> [8] GitHub account: <https://docs.github.com/en/get-started/start-your-journey/creating-an-account-on-github> [9] VS

Code setup: <https://code.visualstudio.com/docs/setup/setup-overview> [10] VS Code with WSL: <https://code.visualstudio.com/docs/remote/wsl> [11] VS Code source-control quickstart: <https://code.visualstudio.com/docs/sourcecontrol/quickstart> [12] VS

Code GitHub integration: <https://code.visualstudio.com/docs/sourcecontrol/github>